

Licence MIAGE

Compte-rendu général

Projet réalisé du 12 Février au 1 juin 2026

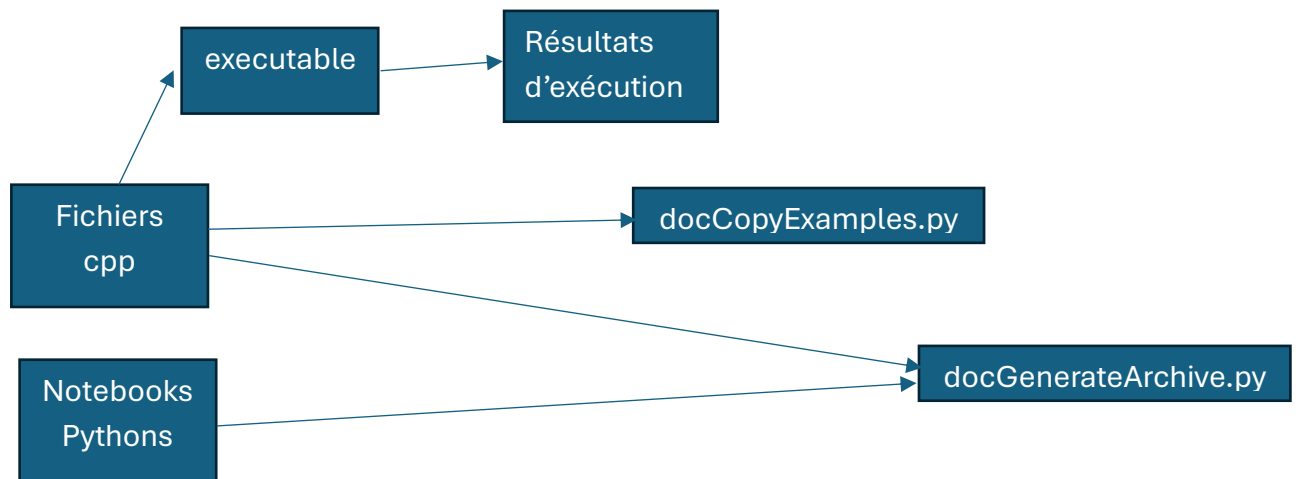
FERRADJ Assia

Introduction

Marmote (MARKovian MOdelling Tools and Environements) integrates est une librairie Conda spécialisée en résolution de modèles de Markov avec variables discrètes. Elle est livrée avec une API C++ et une API Python, qui encapsule les objets C++ .

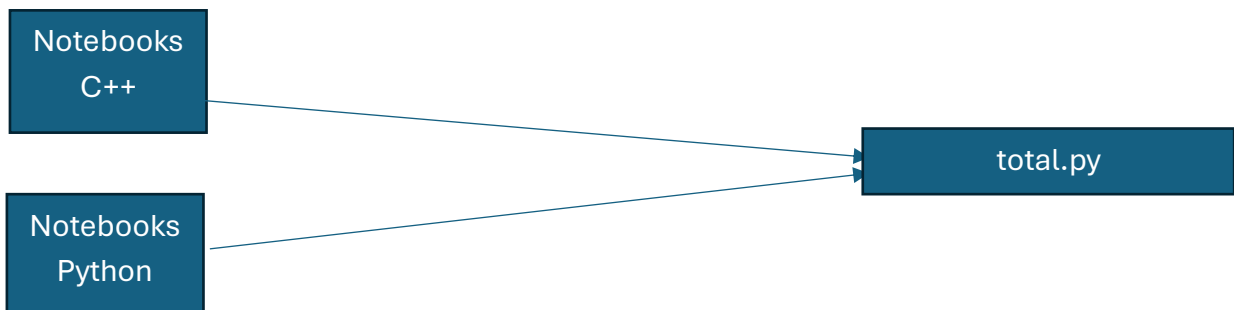
Actuellement, le site de la documentation présente des exemples pour l'utilisation de Marmote, à la fois en Python mais aussi en C++.

Le site web est généré à l'aide de Sphinx, un librairie Python spécialisée dans la construction de documentation. Le workflow actuel repose sur le schéma suivant :



Annexe 1.1 Schéma workflow de base

Le but est de simplifier le schéma de préparation des ressources des exemples, en intégrant des notebooks C++ et d'évoluer vers cette représentation :



Annexe 1.2 Schéma workflow à atteindre

Nous verrons par la suite plus en détail le but de ces deux scripts.

Nous détaillons ci – dessous les objectifs de ce projet :

- De fixer certaines issues
- Optimiser les flux d'exécution
- Unifier le traitement des exemples
- Créer des notebooks C++
- Sécuriser les scripts

Ainsi, nous nous poserons la problématique suivante :

Comment créer des notebooks Jupyter en C++ en toute genericité ?

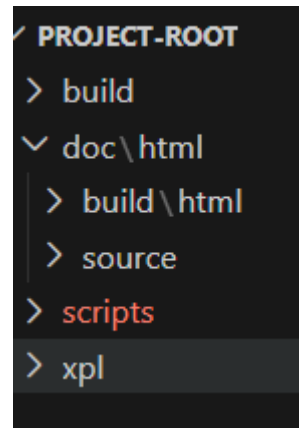
Nous explorerons les différentes solutions disponibles et nous testerons ensuite la meilleure, selon la plateforme que l'on choisit et nous analyserons finalement les limites de cette solution.

Amélioration du flot d'exécution

Au cours de ce projet, nous avons opté pour une démarche agile et itérative, ainsi nous basculerons progressivement, en apportant des résolutions et améliorations petit à petit, vers l'intégration des exemples dans des notebooks c++.

Arborescence

Nous avons reconstruit l'arborescence de notre projet pour qu'elle corresponde à nos besoins.



Arborescence du projet

xpl : contient les exemples au format cpp

build : contient les exe des exemples cpp. Il contient aussi le CMakeLists utile à la compilation des exemples.

doc/html/build : contient les fichiers issus du build Sphinx

doc/html/source : la source avec les fichiers utiles à Sphinx (les notebooks pythons etc...)

scripts : c'est ici que se trouve doc_copyExample.py et doc_generateArchives.py

Lors de la construction de cette arborescence, on a rencontré des issues :

Issue 1 : chemins manquants	Quand on testait les fichiers doc_copyExample, les dossiers xpl et build/bin étaient absents. On avait seulement archive et source.
Issue 2 : noms des exe / pas les bons	Les tableaux corresponding_exe et corresponding_directory ne correspondaient pas avec les exemples obtenus finalement, que l'on voyait dans le dossier media
Issue 3 : marmoteLog qui était absent	marmoteLog n'étant pas considérée comme une cible séparée, cela a

	engendré des erreurs à plusieurs endroits du code
--	---

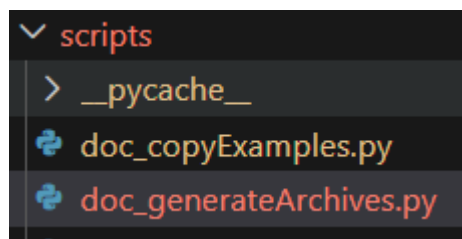
Tableau des issues rencontrées lors de la mise en place de l'arborescence

Préparations de l'environnement de travail

Nous allons, au cours de ce projet, lancer des notebooks en Python et C++ qui utilisent la librairie Marmote, compiler des fichiers C++ et tester la génération du site de la documentation à l'aide de Sphinx. Cela nécessite la création de différents environnements et l'installation de divers outils :

- Python : la version que j'utilise au cours de ce projet est la ...
- MiniConda : j'effectue certaines commandes via l'outil Anaconda Prompt, comme les commandes de création d'environnements

Présentation des scripts



Annexe 2.2.1 Contenu du dossier script

Pour préparer les fichiers d'exemple utiles pour générer le contenu du site web, nous faisons appel à deux fichiers de script :

- **Doc_copyExamples.py** : Ce script automatise la préparation de la documentation Marmote . Il sélectionne des exemples C++ à partir de listes numérotées, exécute leurs programmes correspondants pour générer les fichiers .res et .cmd, copie les fichiers sources .cpp et les résultats dans le répertoire media de la documentation, nettoie les anciens fichiers, et met à jour automatiquement un CMakeLists.txt à partir d'un modèle pour ne compiler que les exemples choisis, avant d'ajouter ce fichier mis à jour dans une archive all_examples.zip.
- **Doc_generateArchives.py** : Ce script automatise toute la préparation des notebooks Marmote pour Google Colab : il copie les notebooks sources, les renomme en versions “_colab”, insère automatiquement des cellules d'installation (condacolab, conda, marmote), remplace les images locales par des URLs du site Marmote, convertit les notebooks en scripts Python si

nécessaire, crée des archives ZIP (pour les notebooks, les versions Python ou les fichiers C++), puis déplace et nettoie les fichiers générés pour produire un ensemble prêt à être publié ou distribué.



8. Création archive C++ (cpp + mcl)

→ cpp_examples.zip

Générer la documentation avec Sphinx

<https://www.sphinx-doc.org/en/master/usage/quickstart.html#running-the-build>

Sphinx est un module Python permettant de générer de la documentation.

Il réutilise les fichiers .rst*, md et notebooks pour les transformer en HTML. Ici, il lit le dossier doc/html/source pour produire les pages.

Il est livré avec un fichier conf.py qui permet de paramétrer l'usage de Sphinx pour notre projet.

*fichier rst : se présente sous cette forme, contient le contenu textuel de la page HTML

```
C++ API
=====

.. toctree::
  :hidden:
  :maxdepth: 2
  :caption: Topics
  :name: cpp_toc

  Commented examples <./cpp_examples>
  Files for download <./cpp_downloads>
  Documentation <./cpp_index>

Documentation and examples to help you build your own ap
using Marmote in C++.

- `Commented examples <./cpp_examples.html>`_
- `Files for download <./cpp_downloads.html>`_
- :doc:`Documentation <./cpp_index>`

Download the :download:`manual <./media/manual.pdf>`.
```

Marmote

C++ API

View page source

C++ API

Documentation and examples to help you build your own application using Marmote in C++.

- Commented examples
- Files for download
- Documentation

Download the manual.

Previous

Next

© Copyright 2023, 2024, 2025 ajm and eh.

Built with Sphinx using a theme provided by Read the Docs.

On a d'abord installé sphinx sur notre environnement avec les extensions suivantes :

```
python -m pip install -U sphinx sphinx-rtd-theme myst-parser nbsphinx
```

Puis on build le projet dans le dossier build/html à partir du dossier source en entrée :

```
python -m sphinx -b html source build\html
```

Issue 1 : extension recommon mark manquante	Build interrompu ; installation de l'extension
Issue 2 : pandoc manquant	Build interrompu : installation via conda forge

Modifications de sécurisation

La sécurisation des scripts s'est faite en deux temps complémentaires.

Dans `doc_generateArchives.py`, on a d'abord renforcé la maîtrise du contexte d'exécution en conservant un contrôle strict du répertoire de lancement (logique `scripts/marmote/source`, sinon arrêt), puis on a ajouté un mode `--dry_run` pour simuler la chaîne sans effets de bord, ce qui permet de vérifier les chemins et la séquence avant toute copie, suppression ou déplacement.

```
# ajout assia : vérification de répertoire
# récupérer le répertoire actuel
current_directory = os.getcwd()
print(f"Working directory is : {current_directory}")
# arrêter si pas dans source, scripts, ou marmote
current_name = os.path.basename(current_directory)
if (current_name=="scripts"):
    position=os.path.join("../", "doc", "html", "source")
    os.chdir(position)
    print("Change directory in : ",os.getcwd())
elif (current_name=="marmote"):
    position=os.path.join("doc", "html", "source")
    os.chdir(position)
    print("Change directory in : ",os.getcwd())
elif (current_name=="source"):
    print("No directory change",os.getcwd())
else:
    print("Wrong working directory. Program stops")
    sys.exit()

# Directories paths
print("Beginning of script")

# ajout assia : dry-run
args = sys.argv
generate_only = False
dry_run = False
if ( len(args) > 1 ):
    if ( args[1] == "-h" ) or ( args[1] == "--help" ):
        print(f"Usage: {args[0]} [-h|--help] [-n|--dry_run]")
        print("\tThis script is to be stored in .../scripts\n\tand is to be run from directory .../doc/html/source")
        sys.exit(0)
    elif ( args[1] == "-n" ) or ( args[1] == "--dry_run" ):
        dry_run = True

# ajout assia : dry-run
if dry_run:
    print("Dry run : not executing archive/copy instructions")
```

Dans `doc_copyExamples.py`, la sécurisation a surtout porté sur la partie la plus sensible, c'est-à-dire l'exécution de commandes externes: on a réaligné les exécutables réellement appelés (notamment en Windows avec suffixe `.exe`), corrigé le mapping des binaires pour éviter les appels erronés, traité le cas particulier d'exemple4 (conflit dossier de données `.mcl` / exécutable), et corrigé des incohérences de destination lors des déplacements de résultats. Le processus a été itératif: diagnostic via `--dry_run`, reproduction des erreurs réelles, correction ciblée, puis validation par exécution complète jusqu'à obtenir un flux stable de génération (`.cpp`, `.res`, `.cmd`, CMake) sans interruption. On a également, comme pour `doc_generateArchive`, arrêté le programme si on se retrouve dans le mauvais dossier.

(chercher : # ajout assia)

```
PS C:\Users\assia\Documents\projet_univ\dossier 2\source\source> python ../../Archive/doc_copyExamples.py --dry_run
>>
Working directory is : C:\Users\assia\Documents\projet_univ\dossier 2\source\source
No directory change C:\Users\assia\Documents\projet_univ\dossier 2\source\source
Beginning of script
Source files destination = C:\Users\assia\Documents\projet_univ\dossier 2\source\source\media [Exists]
CMakeFile destination = C:\Users\assia\Documents\projet_univ\dossier 2\source\source\instructions\all_ex [Exists]
Binary = C:\Users\assia\Documents\projet_univ\dossier 2\source\source\..\..\..\build\bin [Does not exist]
Code = C:\Users\assia\Documents\projet_univ\dossier 2\source\source\..\..\..\xpl [Does not exist]
Dry run: not executing copy instructions
PS C:\Users\assia\Documents\projet_univ\dossier 2\source\source>
```


Présentation de Xeus

xeus est une bibliothèque C++ qui implémente le protocole Jupyter. Elle sert de base pour créer des kernels interactifs, notamment xeus-cling pour exécuter du C++ dans des notebooks cellule par cellule. Concrètement, xeus gère la communication Jupyter (exécution, sortie, complétion), et cling fournit l'interpréteur C++ (LLVM/Clang).

xeus GitHub (core) : <https://github.com/jupyter-xeus/xeus-cling>

Spécifications OS

D'après la documentation officielle de xeus-cling sur GitHub, le projet fournit des paquets prêts à l'emploi pour Linux et macOS, mais pas pour Windows à ce stade. La formulation du README est explicite: les distributions sont disponibles sur Linux/OS X, et il est indiqué qu'aucun package Windows n'est fourni pour le moment. Cela explique les difficultés rencontrées en environnement Windows (conflits de toolchain, intégration plus fragile), alors que le fonctionnement est généralement plus direct sous Linux, notamment via conda-forge et WSL.

Sur Linux et macOS, le fonctionnement de xeus-cling est beaucoup plus fluide parce que les paquets officiels sont fournis pour ces plateformes. En pratique, on installe le kernel via conda-forge dans un environnement dédié, puis Jupyter détecte automatiquement les kernels C++ (xcpp17, xcpp20, etc.) et permet l'exécution cellule par cellule dans le notebook. Le point clé est de garder une chaîne cohérente dans le même environnement (Jupyter, kernel et dépendances C++), ce qui limite les problèmes de linkage ou de kernel crash. C'est précisément ce mode d'usage "packagé" Linux/macOS qui est documenté officiellement dans xeus-cling, contrairement à Windows où aucun package prêt à l'emploi n'est annoncé.

Différents types de bibliothèques Xeus

Xeus se décline sous différentes bibliothèques, l'essentiel est de ne pas les confondre. Seul xeus-cling offre un kernel c++. Sinon, on retrouve :

- Xeus : le cœur du framework, qui implémente jupyter
- Xeus-cling : dont le kernel c++ est basé sur cling
- Xeus-python : qui offre un kernel python
- Xeus-sql : kernel sql
- Xeus-zmq : qui est une couche de transport ZeroMQ qu'utilise les kernels
- Xeus robot, xeus lua ...

La base : Xeus et xeus-zmq. Puis les autres kernels sont construits par-dessus.

On garde en tête que pour nos besoins actuels, on utilise xeus-cling.

Comment créer un notebook c++ avec Xeus

Comme rappelé précédemment, xeus-cling ne propose pas de packages prêt à l'emploi sur Windows. On utilise donc le Windows Subsystem For Linux.

On y installe conda.

On crée un environnement appelé xeus-cpp où on installe marmote xeus-cling et jupyter. On clique sur le lien renvoyé en lançant le serveur jupyter.

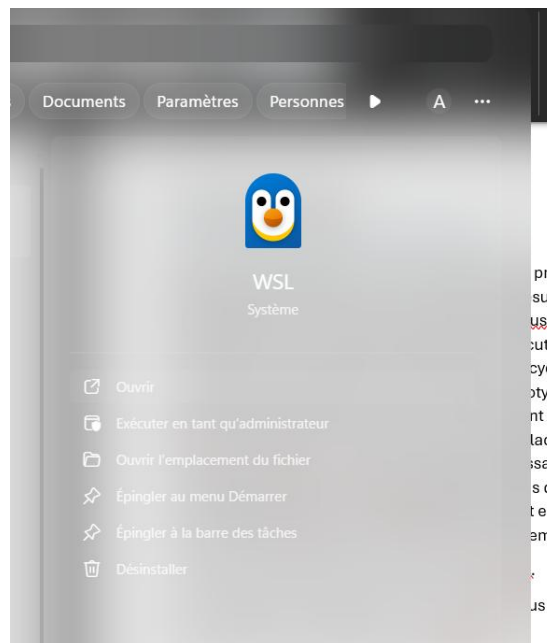
```
conda create -n xeus-cpp-env -c conda-forge python=3.12 jupyterlab xeus-cling -y
```

```
conda activate xeus-cpp-env
```

```
conda install -c marmote -c conda-forge marmote -y
```

```
jupyter kernelspec list
```

```
jupyter lab --no-browser --ip=0.0.0.0 --port=8888
```

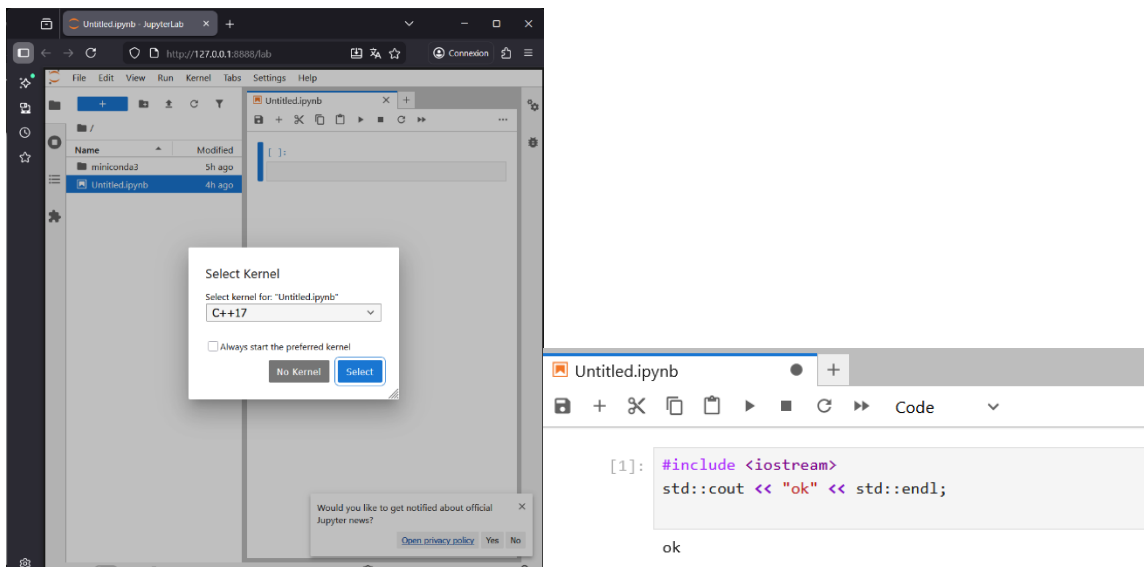


```
(xeus-cpp-env) assia@N15I711-16GR512:~$ jupyter kernelspec list
Available kernels:
python3    /home/assia/miniconda3/envs/xeus-cpp-env/share/jupyter/kernels/python3
xc11       /home/assia/miniconda3/envs/xeus-cpp-env/share/jupyter/kernels/xc11
xc17       /home/assia/miniconda3/envs/xeus-cpp-env/share/jupyter/kernels/xc17
xc23       /home/assia/miniconda3/envs/xeus-cpp-env/share/jupyter/kernels/xc23
xcpp17     /home/assia/miniconda3/envs/xeus-cpp-env/share/jupyter/kernels/xcpp17
xcpp20     /home/assia/miniconda3/envs/xeus-cpp-env/share/jupyter/kernels/xcpp20
xcpp23     /home/assia/miniconda3/envs/xeus-cpp-env/share/jupyter/kernels/xcpp23
```

```
[I 2026-03-05 06:27:43.440 ServerApp] Jupyter Server 2.17.0 is running at:
[I 2026-03-05 06:27:43.440 ServerApp] http://N15I711-16GR512:8888/lab?token=7c2549f264debc176d799628ca7bca
1bad
[I 2026-03-05 06:27:43.440 ServerApp] http://127.0.0.1:8888/lab?token=7c2549f264debc176d799628ca7bcca
ad
[I 2026-03-05 06:27:43.440 ServerApp] Use Control-C to stop this server and shut down all kernels (twice t
ation).
[C 2026-03-05 06:27:43.444 ServerApp]

To access the server, open this file in a browser:
file:/home/assia/.local/share/jupyter/runtime/jpserver-3734-open.html
Or copy and paste one of these URLs:
http://N15I711-16GR512:8888/lab?token=7c2549f264debc176d799628ca7bccab37cc9e5548d1bad
http://127.0.0.1:8888/lab?token=7c2549f264debc176d799628ca7bccab37cc9e5548d1bad
```

Le kernel c++ apparait bien. On teste un exemple simple :



Ca fonctionne. On teste désormais avec un exemple de la doc marmote. On a du supprimé les lignes relatives au main. On a également dû reformulé les entêtes et ajouter les chemins de marmote à la main. Le résultat est satisfaisant.

```
Jupyter Untitled3 Last Checkpoint: 3 minutes ago
File Edit View Run Kernel Settings Help Trusted
+ - X Copy Paste Run Stop Code v JupyterLab C++17

[1]: #pragma clang add_include_path("/home/assia/miniconda3/envs/xeus-cpp-env/include")
#pragma clang add_include_path("/home/assia/miniconda3/envs/xeus-cpp-env/include/marmoteCore")
#pragma clang add_include_path("/home/assia/miniconda3/envs/xeus-cpp-env/include/marmoteMarkovChain")
#pragma clang add_library_path("/home/assia/miniconda3/envs/xeus-cpp-env/lib")
#pragma clang load_library("libmarmoteCore.so")
#pragma clang load_library("libmarmoteMarkovChain.so")

[2]: #include <marmoteMarkovChain/marmoteMarkovChain.h>
#include <marmoteMarkovChain/marmoteSimulationResult.h>
#include <marmoteCore/marmoteDiscreteDistribution.h>
#include <marmoteCore/marmoteSparseMatrix.h>
#include <iostream>
#include <string>

unsigned int n = 10;
double prob1 = 0.1, prob2 = 0.2, prob3 = 0.7;

double states[3] = {0, 1, 2};
double* probas = new double[3];
probas[0] = prob1;
probas[1] = prob2;
probas[2] = prob3;
```

```

discrete sparse
3
0 0 2.500000e-01
0 1 5.000000e-01
0 2 2.500000e-01
1 0 4.000000e-01
1 1 2.000000e-01
1 2 4.000000e-01
2 0 4.000000e-01
2 1 3.000000e-01
2 2 3.000000e-01

stop
discrete values { 0 1 2 } probas { 0.1 0.2 0.7 }
Trajectory
0 1 1
1 2 2
2 2 2
3 1 1
4 0 0
5 0 0
6 1 1
7 0 0
8 1 1
9 2 2
10 2 2

```

Issue 1 : kernel died	Le kernel mourait lors de l'exécution de code Marmote non résolu (headers/libs non configurés).
Issue 2 : version de libboost incohérente	Marmote (ou une dépendance) attendait une version précise (libboost_thread.so.1.82.0). Si elle manque, les programmes ne démarrent pas
Issue 3 : headers marmote introuvables	Headers Marmote introuvables marmoteMarkovChain.h et dépendances non trouvés au départ dans le notebook.

Alternatives à Xeus-cling

Il existe des alternatives à xeus-cling sous windows. On dénote jupyter-cpp-kernel, qui se base sur g++ :

Doc/PyPI : <https://pypi.org/project/jupyter-cpp-kernel/>

GitHub : <https://github.com/shiroinekotfs/jupyter-cpp-kernel>

Guide install Windows : https://raw.githubusercontent.com/shiroinekotfs/jupyter-cpp-kernel/master/INSTALL_ON_WINDOWS.m

On l'a testé sur l'exemple précédent. Cependant, après installation, les bugs s'enchainent au moment de l'exécution.

```
std::cout << std::endl;

delete simRes1;
delete c1;
delete initial;
delete[] probas;
return 0;
}

In file included from c:\mingw\lib\gcc\mingw32\6.3.0\include\c++\random:35:0,
      from C:\Users\assia\miniconda3\envs\jcpp-win\Library\include\marmoteCore\marmotePolicy.h:5,
      from C:\Users\assia\miniconda3\envs\jcpp-win\Library\include\marmoteMarkovChain\marmoteMarkovChain.h:13,
      from C:\Users\assia\AppData\Local\Temp\tmp_un_zefl.cpp:4:
c:\mingw\lib\gcc\mingw32\6.3.0\include\c++\bits\c++0x_warning.h:32:2: error: #error This file requires compiler and library support for the ISO C++ 2011
standard. This support must be enabled with the -std=c++11 or -std=gnu++11 compiler options.
#error This file requires compiler and library support \
^~~~~~

In file included from c:\mingw\lib\gcc\mingw32\6.3.0\include\c++\bits\postypes.h:40:0,
      from c:\mingw\lib\gcc\mingw32\6.3.0\include\c++\iosfwd:40,
      from c:\mingw\lib\gcc\mingw32\6.3.0\include\c++\ios:38,
      from c:\mingw\lib\gcc\mingw32\6.3.0\include\c++\ostream:38,
      from c:\mingw\lib\gcc\mingw32\6.3.0\include\c++\iostream:39,
      from C:\Users\assia\miniconda3\envs\jcpp-win\share\cpp_header\check_cpp.hpp:23,
      from C:\Users\assia\AppData\Local\Temp\tmp_un_zefl.cpp:1:
c:\mingw\lib\gcc\mingw32\6.3.0\include\c++\cwchar:164:11: error: '::vfwscanf' has not been declared
      using ::vfwscanf;
      ^~~~~~
c:\mingw\lib\gcc\mingw32\6.3.0\include\c++\cwchar:170:11: error: '::vswscanf' has not been declared
      using ::vswscanf;
      ^~~~~~
c:\mingw\lib\gcc\mingw32\6.3.0\include\c++\cwchar:174:11: error: '::vwscanf' has not been declared
      using ::vwscanf;
      ^~~~~~
c:\mingw\lib\gcc\mingw32\6.3.0\include\c++\cwchar:191:11: error: '::wcstof' has not been declared
      using ::wcstof;
```

Des versions de g++ incompatibles étaient mises en cause, mais après mise à jour de mingw, récupération des dernières versions de g++ utiles pour l'appel à marmote, le kernel ne fonctionnait plus (il se basait sur un mingwe trop ancien pour marmote).

Cette solution pourrait fonctionner pour créer des notebooks, mais pas pour marmote.

Il existe aussi cppyy , mais nous ne l'avons pas retenu ici, car il s'agit davantage de c++ piloté depuis python, sans kernel, et n'est pas adapté au chargement des headers pour marmote.

Modification du script

J'ai modifié le script de redirection. Si l'on se trouve dans le répertoire du projet, on va remonter à la racine puis rediriger vers doc/html/source. Je place la modification dans les 2 scripts.

```
#Si on n'est pas dans un dossier du projet, on quitte
#Si on se trouve dans source, on poursuit le script
#Sinon, on remonte jusqu'à la racine project-root (à remplacer)
#Puis on redirige vers doc/html/source
if["project-root" in current_directory]:
    if(os.path.basename(current_directory)=="source"):
        print("No directory change needed",os.getcwd())
    else:
        print("Unexpected working directory. Redirecting to the correct location...")
        while(os.path.basename(os.getcwd())!="project-root"):
            print("Moving up...",os.getcwd())
            os.chdir("..")

        path = os.path.join("doc","html","source")
        os.chdir(path)
        print("Redirection completed successfully : ",os.getcwd())
else:
    print("Wrong directory. Program stops")
    sys.exit()
```

Avec quelques tests, cela a fonctionné correctement :

```
PS C:\Users\assia\Documents\projet_univ\dossier 2\project-root\build\bin> python ../../scripts/test.py
Not exactly the location we expected. Redirecting to the good dir ...
Going up ... C:\Users\assia\Documents\projet_univ\dossier 2\project-root\build\bin
Going up ... C:\Users\assia\Documents\projet_univ\dossier 2\project-root\build
Redirecting operation exited successfully : C:\Users\assia\Documents\projet_univ\dossier 2\project-root\doc\html\source
PS C:\Users\assia\Documents\projet_univ\dossier 2\project-root\build\bin> █
```

Test réussi pour le lancement des 2 scripts.

Rédaction des exemples cpp en notebook jupyter

Ici, nous allons recréer les exemples cpp en notebook jupyter. On rajoute les cellules en md, avec le tutoriel, et on détaille un peu plus comme dans les tutos python.

Différence à noter : on ne peut pas passer d'arguments en paramètres, on va donc fixer les valeurs pour nos exemples.

Création d'un script global

On a créé un script global, qui reprend le main des 2 scripts generateArchive et copyExample, ainsi qu'une variable d'environnement qui contient le nom de la racine du projet.

On en a profité pour mutualiser certaines fonctions comme list_and_test , arg_parse(pour prendre en compte l'argument du dry run d'un coup) ou encore stay_or_change_directory qui permet de vérifier que l'on se trouve dans le bon dossier.

Vérification du fichier cmake

Nous testons le fichier CMakeLists fourni dans notre dossier source. Cependant, il semble avoir un problème, car avec find package, Windows recherche un cmake pour marmote, ce qu'il ne trouve pas. Sous Windows, Marmote n'est pas fourni comme un package CMake, donc : pas de marmoteConfig.cmake, pas de Findmarmote.cmake

find_package(marmote) ne fonctionne donc pas. On pourrait utiliser des target_links.

```
CMake Error at CMakeLists.txt:23 (find_package):
  By not providing "Findmarmote.cmake" in CMAKE_MODULE_PATH this project has
  asked CMake to find a package configuration file provided by "marmote", but
  CMake did not find one.


Could not find a package configuration file provided by "marmote" with any
of the following names:

  marmoteConfig.cmake
  marmote-config.cmake

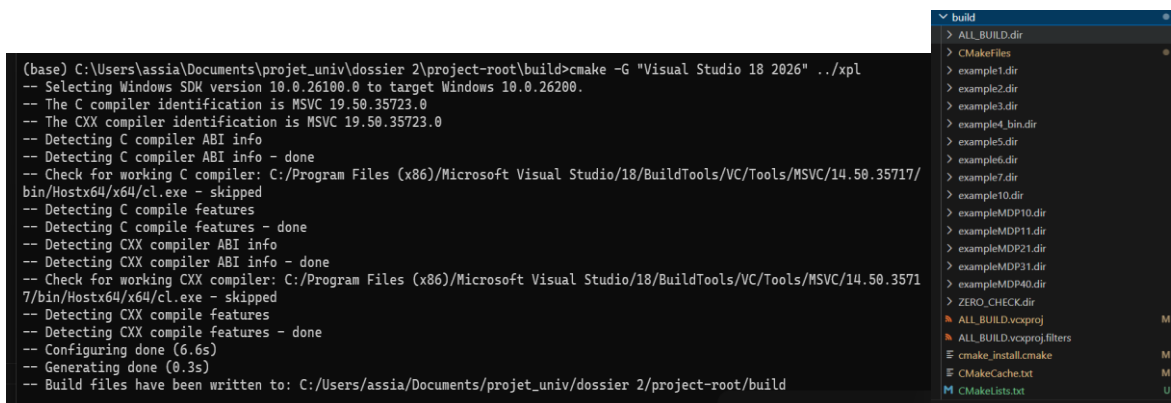

Add the installation prefix of "marmote" to CMAKE_PREFIX_PATH or set
"marmote_DIR" to a directory containing one of the above files.  If
"marmote" provides a separate development package or SDK, be sure it has
been installed.

-- Configuring incomplete, errors occurred!
```

La commande que je vais réaliser est différente, je mets le cmake dans le dossier build à la racine et c'est là que je lance mon cmake. Je le déplace de all_ex vers le dossier build, et je lance la commande en mettant la source xpl :

```
C:\Users\assia\Documents\projet_univ\dossier 2\project-root\build> cmake -G "Visual Studio 18 2026" ../..../xpl
```

Les exe sont bien créés.



Il faut bien veiller à activer le bon environnement conda et ajouter la commande Build

Créer une pile

Nous allons empiler, sur l'environnement marmote-use, l'environnement xeus-cpp-env

```
conda activate marmote-use
```

```
conda activate --stack xeus-cpp-env
```

Vérifier l'installation de Marmote

On a ajouté 2 cellules pour vérifier si marmote est bien installé.

```
# Verify the effective installation of conda
cell_text8 = new_markdown_cell(source="## Verify if marmote is well installed")
cell_text9 = new_code_cell(source="try:\n" \
    "import marmote\n" \
    "    print('Marmote is well installed')\n" \
    "except Exception as e:\n" \
    "    print('Marmote is not available.')\n" \
    "    print(e)")

# Insert the cells at the beginning of the notebook
notebook["cells"] = [
    cell_text1,
    cell_text2,
    cell_text3,
    cell_code4,
    cell_text5,
    cell_code6,
    cell_text7,
    cell_text8,
    cell_text9
] + notebook["cells"]
```

Sinon, on propose l'exécution de la cellule de code avec le patch et on met les indications nécessaires

Transcription des fichiers cpp en notebook jupyter

Pendant mes transcriptions de fichiers, j'ai dénoté plusieurs problèmes :

- Une cellule d'erreur peut casser le kernel. Il faut donc redémarrer le kernel pour éviter de voir de nouvelles erreurs apparaître.
- il y a parfois des problèmes d'include, il faut charger les bonnes librairies avec pragma

Outre ces problèmes, j'ai également eu besoin de rédiger les cellules explicatives de code. Pour cela, je me suis inspirée de la section Exemple avec les notebook Python.

J'ai ensuite rédigé un prompt pour un agent Codex afin d'améliorer la précision et la fidélité de conversion en notebook c++ à partir des 2 sources de données : le notebook python et les fichiers d'exemple c++. J'ai également fait le choix de masquer les cellules qui n'apportent pas d'instructions pertinentes avec un tag hidden. Voici le prompt :

```
Tu es un expert Marmote, C++, Jupyter et documentation scientifique.

Ta mission :
À partir d'un notebook Python Marmote (MDPLesson), tu dois produire une version **C++** qui soit la
traduction la plus fidèle possible du notebook Python.

Correspondances officielles entre les notebooks Python et les exemples C++ :
- MDPLesson1 (Python) → MDPEXample10 (C++)
- MDPLesson2 (Python) → MDPEXample11 (C++)
- MDPLesson3 (Python) → MDPEXample21 (C++)
- MDPLesson4 (Python) → MDPEXample31 (C++)
- MDPLesson5 (Python) → MDPEXample41 (C++)

Organisation des fichiers :
- Les notebooks Python originaux se trouvent dans : instructions/
- Les fichiers C++ officiels correspondants se trouvent dans : xpl/
Tu dois t'appuyer sur ces deux sources pour garantir une traduction fidèle.
```

```
Contraintes de fidélité :
1. Tous les commentaires doivent être pédagogiques, comme dans les notebooks Python.
2. Le style doit être identique aux notebooks Python :
- phrases explicatives avant chaque bloc
- commentaires dans le code
- affichages lisibles
3. Si une fonction Python n'existe pas en C++, tu dois :
- proposer l'équivalent C++
- expliquer brièvement la différence
- adapter le code pour obtenir le même résultat logique.

Contraintes Marmote :
4. Ne pas utiliser les fonctions obsolètes :
- solpol (n'existe plus)
- writeSolution() (bug)
5. Utiliser writeSolutionByDim#() si nécessaire.
6. Toujours vérifier la cohérence avec les fichiers C++ officiels dans xpl/.

Entrée :
Je te fournis ci-dessous le contenu du notebook Python Marmote (copié depuis instructions/).

Sortie attendue :
Un notebook complet, alternant cellules Markdown et cellules C++, qui traduit fidèlement le notebook
Python en C++, prêt à être intégré dans la documentation.
```

```
Objectif :
Créer un notebook C++ qui soit **strictement parallèle** au notebook Python correspondant :
- mêmes sections
- mêmes titres
- mêmes explications textuelles
- mêmes étapes de construction du modèle
- mêmes affichages
- mêmes résultats (autant que possible)

Structure attendue du notebook C++ :
- Cellule Markdown : titre du notebook
- Cellule Markdown : introduction
- Cellule C++ : includes Marmote + initialisations
- Cellule Markdown : "Définition du modèle"
- Cellule C++ : création des états, actions, matrices, distributions
- Cellule Markdown : "Résolution du MDP"
- Cellule C++ : appel aux méthodes de résolution (PolicyIteration, ValueIteration, etc.)
- Cellule Markdown : "Analyse des résultats"
- Cellule C++ : affichage des politiques, valeurs, trajectoires
```

Pour générer la documentation à partir de la nouvelle méthode, j'ai ajusté le fichier `example_cpp.rst` pour qu'il ressemble davantage à celui en python. Je suis passée de l'utilisation du dossier `example` au dossier `cpptutos` qui contient les notebook c++.

Voici la nouvelle rédaction du fichier :

```
C++ Tutorial and Examples
-----

.. toctree::
    :maxdepth: 2
    :caption: Table of Examples
    :name: cpexampletoc
    :hidden:

    cpptutos/Lesson1.ipynb
    cpptutos/Lesson2.ipynb
    cpptutos/Lesson3.ipynb
    cpptutos/Lesson4.ipynb
    cpptutos/Lesson5.ipynb
    cpptutos/Lesson6.ipynb
    cpptutos/Lesson7.ipynb
    cpptutos/MDP_Lesson1.ipynb
    cpptutos/MDP_Lesson2.ipynb
    cpptutos/MDP_Lesson3.ipynb
    cpptutos/MDP_Lesson4.ipynb
    cpptutos/MDP_Lesson5.ipynb

**Markov Chain Lessons**

1. `Lesson 1: <./cpptutos/Lesson1.html>`_ making Markov chains
2. `Lesson 2: <./cpptutos/Lesson2.html>`_ solving Markov chains
3. `Lesson 3: <./cpptutos/Lesson3.html>`_ working with state spaces
4. `Lesson 4: <./cpptutos/Lesson4.html>`_ working with distributions
5. `Lesson 5: <./cpptutos/Lesson5.html>`_ predefined Markov chains
6. `Lesson 6: <./cpptutos/Lesson6.html>`_ manipulating sets and state spaces
7. `Lesson 7: <./cpptutos/Lesson7.html>`_ structural analysis of Markov chains
8. `Lesson 10: <./cpptutos/Lesson10.html>`_ multidimensional random walks on a large grid
```

VS l'ancienne version :

C++ Tutorial and Examples

```
.. toctree::
    :maxdepth: 2
    :caption: Table of Examples
    :name: cppexampletoc
    :hidden:
```

```
example/example1
example/example2
example/example3
example/example4
example/example5
example/example6
example/example7
example/example10
example/exampleMDP10
example/exampleMDP11
example/exampleMDP21
example/exampleMDP31
example/exampleMDP40
```

The following examples show elementary manipulations of Marmote objects:

```
* :doc:`Example 1 <./example/example1>`: create a Markov Chain and perform Monte-Carlo simulation
* :doc:`Example 2 <./example/example2>`: create a Markov Chain and compute transient distributions
* :doc:`Example 3 <./example/example3>`: as in Example 2 with several chains
* :doc:`Example 4 <./example/example4>`: read and write Markov Chains from/to files
* :doc:`Example 5 <./example/example5>`: create a Markov Chain and compute the stationary distribution with different methods
* :doc:`Example 6 <./example/example6>`: manipulate Sets (state spaces)
* :doc:`Example 7 <./example/example7>`: create Markov Chains and perform structural analysis
```

On ne se base désormais plus sur les fichiers rst du dossier example. J'ai donc enlevé les attributs :doc.

Je me mets dans l'anaconda prompt. J'active l'environnement marmote-use dans lequel j'ai également installé sphinx et pandoc (on note qu'on aurait également pu créer un environnement sphinx et faire la commande `--stack` pour empiler les 2 environnements). De cette manière, je lance la commande précédente depuis le dossier doc/html :

```
python -m sphinx -b html source build\html
```

Tu es un expert Marmote, Xeus-cling, C++, Jupyter et documentation scientifique.

Ta mission :

Générer un notebook Jupyter **C++ (Xeus-cling)** qui reproduit **fidèlement** la structure, les explications, les commentaires, les étapes et la pédagogie des notebooks Python Marmote (MDP21, MDP22, MDP23, etc.).

IMPORTANT — Organisation des fichiers :

- Les notebooks Python originaux se trouvent dans le dossier : pytutos/ et les notebooks cpp dans le dossier : cpptutos
- Les fichiers C++ sources correspondants se trouvent dans le dossier : xpl/

Tu dois t'appuyer sur ces deux sources pour garantir une conversion fidèle.

Objectif :

Créer un notebook C++ qui soit **strictement parallèle** au notebook Python correspondant :

- mêmes sections
- mêmes titres
- mêmes explications textuelles
- mêmes étapes de construction du modèle
- mêmes affichages
- mêmes résultats (autant que possible)
- mêmes graphiques si présents (ou équivalent texte si non faisable)

Contraintes techniques :

1. Le code doit être **100% exécutable** dans Xeus-cling.

2. Tous les includes Marmote doivent être corrects :

```
#pragma cling add_include_path(...)

#pragma cling add_library_path(...)

#pragma cling load("libmarmoteCore.so")

#pragma cling load("libmarmoteMarkovChain.so")
```

3. Le notebook doit être structuré comme un notebook Python :

- Cellule Markdown : titre du notebook
- Cellule Markdown : introduction
- Cellule C++ : imports + pragma cling
- Cellule Markdown : "Définition du modèle"
- Cellule C++ : création des états, actions, matrices, distributions
- Cellule Markdown : "Résolution du MDP"
- Cellule C++ : appel à PolicyIteration / ValueIteration
- Cellule Markdown : "Analyse des résultats"
- Cellule C++ : affichage des politiques, valeurs, trajectoires

Contraintes de fidélité :

4. Tous les commentaires doivent être pédagogiques, comme dans les notebooks Python.

5. Le style doit être **identique** aux notebooks Python Marmote :

- phrases explicatives avant chaque bloc
- commentaires dans le code
- affichages lisibles

6. Si une fonction Python n'existe pas en C++, tu dois :

- proposer l'équivalent C++
- expliquer la différence
- adapter le code pour obtenir le même résultat

7. Tu dois éviter les fonctions obsolètes de Marmote :

- ne pas utiliser `solpol` (n'existe plus)
- ne pas utiliser `writeSolution()` (bug)
- utiliser `writeSolutionByDim#()` si nécessaire

8. Le notebook doit être ****propre, clair, académique****, prêt à être intégré dans une documentation Sphinx.

Entrée :

Je te fournis le notebook Python Marmote correspondant (copié ci-dessous depuis instructions/).

Tu peux également consulter le fichier C++ original correspondant dans `xpl/` si nécessaire.

Sortie attendue :

Un notebook complet, alternant cellules Markdown et cellules C++, prêt à être collé dans un fichier `.ipynb`.

IMPORTANT :

- Ne jamais inventer de fonctions Marmote qui n'existent pas.
- Toujours vérifier la cohérence avec les exemples C++ officiels dans `xpl/`.
- Toujours commenter chaque étape comme dans les notebooks Python.